runs the experiment twice: first without the peek *(peek* is False) and then with it (*peek* is True). Line 5 creates one qubit and two classical bits. Lines 6 and 9 are the two Hadamard gates. Lines 7 and 8 record a mid-circuit measurement in classical bit 0, but only when *peek* is True. Line 10 records the final measurement in classical bit 1. Lines 11 to 13 run 1000 shots and print the counts, just as in *qiskit13.py.* Remember the bit-ordering convention from the previous article: Qiskit prints classical bit 0 on the right, so in each result the right-hand character is the peek and the left-hand character is the final outcome. Run the program with the command *python qiskit14.py*. The output is shown in Figure 4.

```
(qiskit_env) # python qiskit14.py
No peeking: {'00': 1000}
Peeking: {'11': 281, '01': 231, '00': 239, '10': 249}
```

Figure 4: Output of the program *qiskit14.py*

Without peeking, every shot gives '00': the final outcome is 0, exactly as predicted, and the right-hand 0 is simply the unused classical bit. With peeking, all four two-bit results appear roughly 250 times each, because the peek is random and so is the final outcome. Merely looking at the qubit halfway through has destroyed the interference: after the mid-circuit measurement, the qubit is definitely in $|0\rangle$ or in $|1\rangle$; only one route remains, and there is nothing for it to cancel against. Physicists say that the measurement has made which-path information available, which destroys interference; the double-slit experiment in physics behaves likewise when detectors watch the slits.

On a real processor, interference can be lost even when nobody peeks. Some of the hardware errors we met in the previous article, such as dephasing, scramble the relative phase on which interference depends, and the qubit then behaves just like the one we peeked at: probabilities add, and nothing cancels. It is as if the environment was peeking at our qubits, although that is only an analogy.

Myth busting: Trying all answers at once

Popular accounts often claim that a quantum computer is fast because, after $H^{\otimes n}$, its register holds all 2^n inputs at once and evaluates a function on all of them in parallel. The first half is true; the conclusion is not. Measuring such a register yields a single random basis state, no better than a guess: the 2^n amplitudes genuinely exist, but we can never read them all out. A quantum algorithm succeeds only by choreographing interference, so that the amplitudes of wrong answers cancel and those of right answers reinforce one another before the measurement. Even that is no magic wand: it works only when a problem has structure that a clever circuit can exploit, which is why quantum computers are expected to speed up certain problems dramatically, others modestly, and many not at all. The computer scientist Scott Aaronson has long carried this disclaimer at the top of his blog: "If you take nothing else from this blog: quantum computers won't solve hard problems instantly by just trying all solutions in parallel." Parallelism sets the stage; interference performs the play.

How the hardware sees a Hadamard gate

In the previous article we learned that IBM's Heron processors support only a short list of native gates: X, SX, RZ and CZ. So how does the chip perform a Hadamard gate? The *qiskit-ibm-runtime* package, which we installed in the fourth article, includes fake backends: offline models of real IBM processors, stored with their native gates, qubit connectivity and a snapshot of their calibration data. The program *qiskit15.py*, shown below, transpiles a single Hadamard gate for *FakeMarrakesh*, a model of *ibm_marrakesh*, the 156-qubit Heron R2 processor we used in the fourth article. Nothing is sent to IBM and no API key is needed: the program only translates the circuit; it does not run it. For once, we skip our small tradition of sending code to a real quantum computer; to see how a gate is translated, a model of the processor is all we need.

```
1.  from qiskit import QuantumCircuit, transpile
2.  from qiskit_ibm_runtime.fake_provider import
    FakeMarrakesh
3.  backend = FakeMarrakesh( )
4.  qc = QuantumCircuit(1, 1)
5.  qc.h(0)
6.  qc.measure(0, 0)
7.  transpiled = transpile(qc, backend, initial_layout=[0])
8.  print(transpiled.draw( ))
9.  sx = backend.target["sx"][(0,)]
10. print(f"{backend.name}, qubit 0: SX {sx.
    duration*1e9:.0f} ns, error {sx.error:.1e}")
```

Line 7 places our qubit on physical qubit 0 and rewrites the circuit using native gates. The output, shown in Figure 5, reveals that the Hadamard gate becomes RZ(π/2), then SX, then RZ(π/2) again, together with a global phase of π/4, which, as we have seen, changes no measurement probability. Better still, RZ gates on IBM hardware are virtual: instead of sending a pulse to the qubit, the control system simply shifts the phase reference of every subsequent pulse in software. The calibration snapshot in the version of *qiskit-ibm-runtime* used for this article (0.48.0) lists RZ with zero duration and zero error, while SX on qubit 0 takes 36 nanoseconds with an error rate of about 1.3 in 10,000. Lines 9 and 10 print the stored SX values, as the last line of Figure 5 shows; another version of the package may store different ones. So, on this processor model, the Hadamard gate costs a single physical microwave pulse, the SX gate, sandwiched between two virtual RZ gates that exist only as bookkeeping in the control software. Now add two more lines,